Projet final Python · ING3

Moteur de traitement et d'analyse de logs

Rapport final

Équipe

Novembre 2026

1. Résumé

Ce projet est un programme en ligne de commande qui lit un très gros fichier de logs, le nettoie, trie chaque ligne selon son type, calcule des statistiques et détecte des attaques. Il se lance avec une seule commande :

```
python main.py --log-file=chemin/vers/logs.log
```

Nous l'avons testé sur un fichier unique de **15,2 Go**, soit 111 047 646 lignes de 16 types différents, que nous avons généré nous-mêmes. Le programme le traite en **62 minutes** avec un pic de RAM de **219 Mo**.

15,2 Go	219 Mo	2,0 %	100 %
dans un seul fichier	pic de RAM	lignes rejetées	attaques retrouvées

Trois résultats à retenir :

• **La mémoire ne suit pas la taille du fichier.** Le programme ne garde qu'un lot de 10 000 lignes à la fois. La RAM augmente surtout avec le nombre d'adresses IP et d'utilisateurs différents.

• **Le nettoyage est vérifié à la ligne près.** Le générateur compte chaque défaut qu'il injecte. Les 6 496 754 réparations et les 2 204 628 rejets du pipeline correspondent exactement à ces comptes.

• **La détection retrouve toutes les attaques injectées.** Le rappel est de 100 % sur les dix scénarios. La seule erreur vient des comptes de service bloqués, signalés à tort comme force brute (section 6).

2. Architecture

Le code est un package Python, `logengine`, piloté par `main.py`. Chaque ligne descend le pipeline une par une. Seule la fin du pipeline regroupe les enregistrements par lots de 10 000, pour les analyser ensemble.

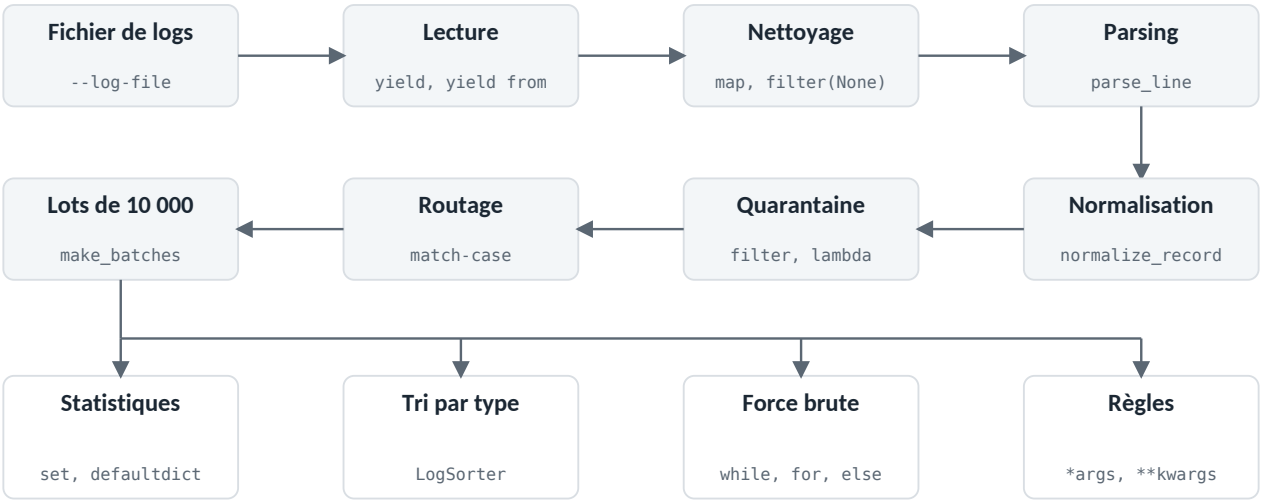


Figure 1. Parcours d'une ligne, du fichier jusqu'aux analyses.

Fichier	Rôle	Module de l'énoncé
main.py	Ligne de commande, lecture des options	Point d'entrée
ingestion.py	Lecture avec yield et yield from	1
parsing.py	Ligne de texte vers dictionnaire	1
transform.py	Nettoyage, normalisation, routage	2
aggregation.py	Compteurs et ensembles d'IP	3
bruteforce.py	Détection de la force brute	3
rules.py	Moteur de règles de sécurité	4
audit.py	Rapport sur les variables en mémoire	4
sorting.py	Tri des lignes dans un fichier par type	Consigne du professeur
pipeline.py	Enchaînement de toutes les étapes	Tous
generator.py	Création du fichier de logs	Consigne du professeur
evaluation.py	Comparaison avec la vérité terrain	Validation
charts.py	Graphiques avec matplotlib	Rapport

3. Jeu de données

Les consignes demandent un seul fichier de 15 Go, avec des logs structurés et très variés : machines, connexions, actions, routeurs. Il doit aussi contenir des erreurs et des lignes malformées à nettoyer. Notre générateur écrit tout dans un seul fichier, dans l'ordre du temps, sur 30 jours. Le fichier final fait 15,2 Go tel qu'affiché par Windows, soit 16,3 milliards d'octets.

Format des lignes

Chaque ligne suit le même format à six champs, séparés par le caractère | :

```
horodatage | niveau | ip:port/proto | utilisateur | service | payload
```

Le payload prend les trois formes demandées par l'énoncé :

```
2026-10-05T08:14:03.120Z | WARNING | 10.20.3.8:51234/TCP | u00042 | auth |
    {"event":"login_failed","code":401,"method":"password"}

2026-10-05T08:14:03.480Z | WARNING | 91.12.4.7:40112/TCP | - | firewall |
    conn=(172.16.0.10,3389,TCP) action=DENY bytes=60

2026-10-05T08:14:04.002Z | ERROR | 10.255.6.1:514/UDP | - | router |
    %LINK-3-UPDOWN: Interface GigabitEthernet0/4, changed state to down
```

Variété des données

Famille	Types de logs	Contenu
Réseau	router, switch, firewall, wifi, dns, dhcp, vpn	Interfaces, voisins OSPF et BGP, connexions, bornes wifi, requêtes DNS, baux DHCP, sessions VPN
Connexions	auth, web	Connexions réussies et échouées, requêtes HTTP
Actions	action	Fichiers ouverts ou supprimés, sudo, clés USB, installations
Machines	machine, system, antivirus	CPU, RAM, disque, température, messages système, menaces
Applications	app, database, mail	Erreurs applicatives, requêtes lentes, envois de mails

Le trafic suit les heures de bureau, baisse le week-end et contient deux incidents : une panne le 10 octobre et un pic web le 18 octobre.

Défauts injectés

Environ 9 % des lignes reçoivent un défaut. Le fichier `manifest.json` enregistre le nombre exact de chaque défaut, ce qui sert de vérité terrain en section 5.

Famille	Exemples	Traitement
À réparer	Niveaux err, WARN, fatal. Dates en français, en secondes, avec fuseau. IP 010.001.002.003 ou [10.1.2.3]. Couleurs ANSI, BOM. JSON avec guillemets simples ou virgule en trop.	Corrigé
Bruit	Lignes vides, commentaires logrotate, doublons	Retiré
Invalide	Ligne coupée, CORRUPTED_LINE_WITHOUT_FORMAT, champ manquant, IP 999.12.3.4, port 70000, mois 13, année 1970, JSON tronqué	Rejeté et compté
Ancien format	Lignes au format du TP6, avec le statut HTTP mais sans niveau	Converti

Scénarios d'attaque

Chaque journée contient des attaques planifiées, dont les adresses sont listées dans le manifeste : force brute rapide et lente, connexion réussie après une rafale, pulvérisation de mots de passe, scan de ports, injections web, exfiltration, voyage impossible sur le VPN, routeur reconfiguré par un compte inconnu et adresses en liste noire. Nous avons aussi ajouté du trafic trompeur : des comptes de service au mot de passe expiré et un scanner interne autorisé.

4. Implémentation

Ligne de commande

Le programme se pilote uniquement par des options. `sys.argv` contient les mots tapés après le nom du script, et `parse_options` les transforme en dictionnaire.

```
def parse_options(arguments: list) -> dict | None:
    options = {"log-file": []}
    for argument in arguments:
        name, _, value = argument.removeprefix("--").partition("=")
        if not argument.startswith("--"):
            return None
        if name == "log-file":
            options["log-file"].append(value)
        elif name in KNOWN_OPTIONS:
            options[name] = value
        else:
            return None
    return options
```

Module 1 : ingestion

`read_log_file` rend une ligne à chaque appel et garde le fichier ouvert entre deux lignes. `read_sources` délègue à ce générateur avec `yield from`. Notre fichier est unique, mais l'énoncé demande de savoir enchaîner plusieurs sources : il suffit de répéter `--log-file`.

```
def read_log_file(path: str) -> Iterator[str]:
    # errors="replace" : un octet illisible ne doit pas arrêter la lecture.
    with open(path, "r", encoding="utf-8", errors="replace") as file:
        for line in file:
            yield line

def read_sources(paths: list[str]) -> Iterator[str]:
    for path in paths:
        try:
            yield from read_log_file(path)
        except FileNotFoundError:
            print(f"Fichier introuvable, ignoré : {path}")
        except OSError as error:
            print(f"Lecture impossible, fichier ignoré : {error}")
```

`parse_line` transforme chaque ligne en dictionnaire. Après normalisation, les métadonnées fixes sont rangées dans un tuple : `record["meta"] = (ip, port, protocole)`. Un tuple est immuable : ces valeurs ne peuvent plus changer par erreur.

Module 2 : transformation

Le pipeline s'écrit comme une chaîne de `map` et `filter`. Ces fonctions sont paresseuses : aucune ligne n'est lue tant que la boucle finale ne la demande pas. Voici le début de la chaîne :

```
read = count_stage(lines, "read", quality)
cleaned = map(lambda line: sanitize_line(line, quality), read)
# filter(None, ...) retire les chaînes vides.
not_empty = count_stage(filter(None, cleaned), "not_empty", quality)
data_lines = count_stage(
    filter(lambda line: not line.startswith(("#", "-", "=")), not_empty),
    "data_lines",
    quality,
)
unique = drop_duplicates(data_lines, quality)
parsed = map(lambda line: parse_line(line, quality), unique)
normalized = map(lambda rec: normalize_record(rec, quality), parsed)
```

Le routage remplace une longue suite de `if/elif`. Chaque `case` décrit la forme du payload : dictionnaire, tuple de connexion ou chaîne brute.

```
def route_record(record: dict) -> dict:
    match record["payload"]:
        case {"path": "/health"}:
            route = "healthcheck"
        case {"level": "ERROR" | "CRITICAL", "code": int()}:
            route = "app_error"
        case {"status": status} if status >= 500:
            route = "http_5xx"
        case {"status": status} if status >= 400:
            route = "http_4xx"
        case {"status": _}:
            route = "http_ok"
        case {"event": "login_failed" | "login_success" | "logout"}:
            route = "auth_event"
        case {"threat": _}:
            route = "threat"
        case {"event": _, "target": _}:
            route = "user_action"
        case {"event": _}:
            route = "session_event"
        case {"cpu": _}:
            route = "machine_metric"
        case dict():
            route = "json_other"
        case (_, port, _) if port in SENSITIVE_PORTS:
            route = "connection_sensitive"
        case (_, _, _):
            route = "connection"
        case str() as text if text.startswith("%"):
            route = "network_device"
        case str():
            route = "raw_text"
        case _:
            route = "unknown"
    record["route"] = route
    return record
```

Tri par type

Chaque ligne valide est réécrite proprement dans le fichier de son service. L'écriture se fait au fil de la lecture : rien n'est gardé en mémoire. Un service inconnu part dans `other.log`, car un texte lu dans un log ne doit jamais choisir un nom de fichier.

```
def write(self, batch: list) -> None:
    for record in batch:
        service = record["service"]
        if service not in KNOWN_SERVICES:
            service = "other"
        if service not in self.files:
            path = os.path.join(self.folder, f"{service}.log")
            self.files[service] = open(path, "w", encoding="utf-8")
        self.files[service].write(format_record(record) + "\n")
        self.counts[service] += 1
```

Module 3 : agrégation et force brute

Les IP actives et les IP en erreur sont stockées dans des `set`. La liste noire est un `frozenset` : elle ne peut pas être modifiée pendant l'analyse. Sur le jeu complet, l'intersection contient 360 IP sur 600 en liste noire.

```
def compare_with_blacklist(
    active: set, errors: set, blacklist: frozenset
) -> dict:
    common = active & blacklist
    return {
        "active": len(active),
        "with_errors": len(errors),
        "blacklist": len(blacklist),
        "union": len(active | blacklist),
        "intersection": len(common),
        "difference": len(active - blacklist),
        "symmetric_difference": len(active ^ blacklist),
        "errors_and_blacklist": len(errors & blacklist),
        "blacklisted_active_ips": sorted(common),
    }
```

Le détecteur de force brute combine toutes les instructions de contrôle demandées. `continue` saute les lignes hors sujet. Si trop d'alertes s'accumulent, `break` quitte le `for` pour les publier, et le `while` reprend à la ligne suivante. Le `else` du `for` s'exécute seulement sans `break` : il valide un cycle complet.

```
def run_cycle(self, batch: list, publish: Callable) -> None:
    position = 0
    while position < len(batch):
        alerts = []
        for index in range(position, len(batch)):
            record = batch[index]
            if record["service"] != "auth" or record["user"] is None:
                continue
            if not isinstance(record["payload"], dict):
                continue
            event = record["payload"].get("event")
            key = (record["ip"], record["user"])
            if event == "login_failed":
                alert = self.on_failure(key, record["timestamp"])
            elif event == "login_success":
                alert = self.on_success(key, record["timestamp"])
            else:
                continue
            if alert is None:
                continue
            alerts.append(alert)
            if len(alerts) >= MAX_ALERTS_PER_PASS:
                position = index + 1
                self.interrupted_passes += 1
                break
        else:
            position = len(batch)
            self.completed_cycles += 1
            self.forget_old_keys(batch[-1]["timestamp"])
        publish(alerts)
```

Module 4 : règles et audit

`apply_security_rules` reçoit les règles dans `*args` et la configuration dans `**kwargs`. Les règles sont des fonctions ordinaires, passées en argument. Si l'une d'elles plante, l'erreur est comptée et l'analyse continue.

```
def apply_security_rules(*args: Callable, **kwargs: Any) -> list:
    records = kwargs.pop("records")
    alerts = []
    for rule in args:
        for record in records:
            try:
                alert = rule(record, **kwargs)
            except (KeyError, TypeError, AttributeError, ValueError):
                kwargs["errors"][rule.__name__] += 1
                continue
            if alert is not None:
                alerts.append(alert)
    return alerts
```

L'appel dans le pipeline montre la configuration dynamique. Pour changer un seuil, il suffit d'ajouter un argument nommé, par exemple `scan_ports=30`.

L'audit appelle `globals()` et `locals()` à la fin de `run_pipeline`, puis écrit `audit.json` : type, taille et aperçu de chaque variable. Il n'itère jamais sur les générateurs, car cela les viderait. Les variables dont le nom contient *password*, *secret* ou *token* sont masquées.

5. Résultats du nettoyage et du tri

Le tableau suivant suit le nombre de lignes à chaque étape du pipeline. Les sondes de supervision `/health` sont retirées à la fin, car elles ne disent rien de l'activité réelle.

Étape	Lignes	Part des lignes lues
Lues	111 047 646	100,0 %
Non vides	110 567 302	99,6 %
Hors commentaires	110 361 633	99,4 %
Valides	107 812 698	97,1 %
Analysées	107 282 327	96,6 %

Le pipeline a lu exactement les 111 047 646 lignes écrites par le générateur. Les tableaux ci-dessous comparent ses compteurs au manifeste. Un écart de 0 signifie que chaque défaut injecté a été retrouvé, et rien d'autre.

Réparations

Compteur	Injecté	Observé	Écart
Horodatages	1 235 594	1 235 594	0
Niveaux	2 127 969	2 127 969	0
Adresses IP	813 377	813 377	0
Protocoles	411 469	411 469	0
Utilisateurs	322 263	322 263	0
JSON réparés	367 633	367 633	0
Caractères cachés	548 266	548 266	0
Doublons	344 307	344 307	0
Ancien format	265 987	265 987	0
Secrets masqués	59 889	59 889	0

Rejets

Compteur	Injecté	Observé	Écart
Structure	959 130	959 130	0
IP invalide	274 636	274 636	0
Port	205 611	205 611	0
Date illisible	274 497	274 497	0
Date hors plage	136 905	136 905	0
Niveau inconnu	137 588	137 588	0
JSON irrécupérable	147 377	147 377	0
Service	68 884	68 884	0

Résultat du tri

Les 107 282 327 lignes analysées sont rangées dans 16 fichiers, un par type de log, et les 2 204 628 lignes invalides dans `bad_lines.log`, avec le motif de chaque rejet.

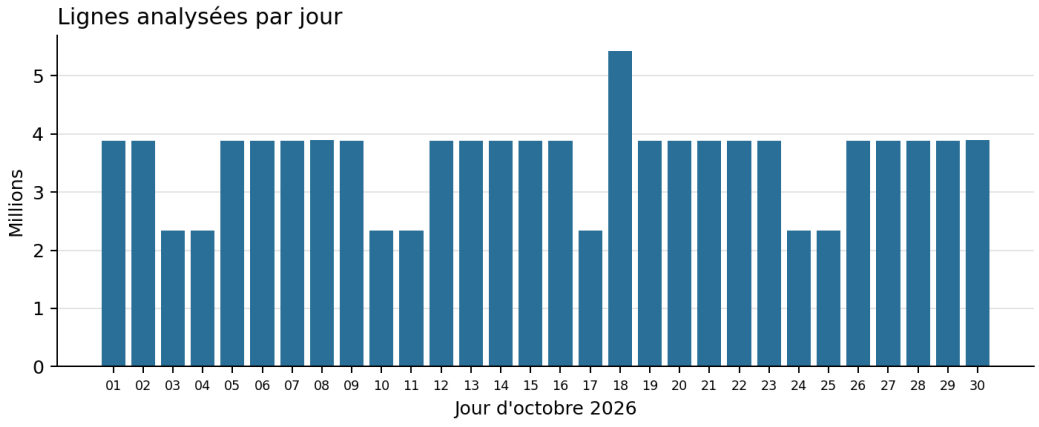
Type de log	Lignes triées	Part
web.log	25 358 986	23,6 %
firewall.log	15 679 784	14,6 %
dns.log	10 025 292	9,3 %
auth.log	8 920 417	8,3 %
router.log	6 854 435	6,4 %
action.log	6 679 561	6,2 %
app.log	5 584 368	5,2 %
switch.log	4 892 779	4,6 %
wifi.log	4 461 353	4,2 %
database.log	3 914 441	3,6 %
dhcp.log	3 345 822	3,1 %
mail.log	3 341 713	3,1 %
vpn.log	3 335 213	3,1 %
system.log	1 957 902	1,8 %
machine.log	1 952 850	1,8 %
antivirus.log	977 411	0,9 %

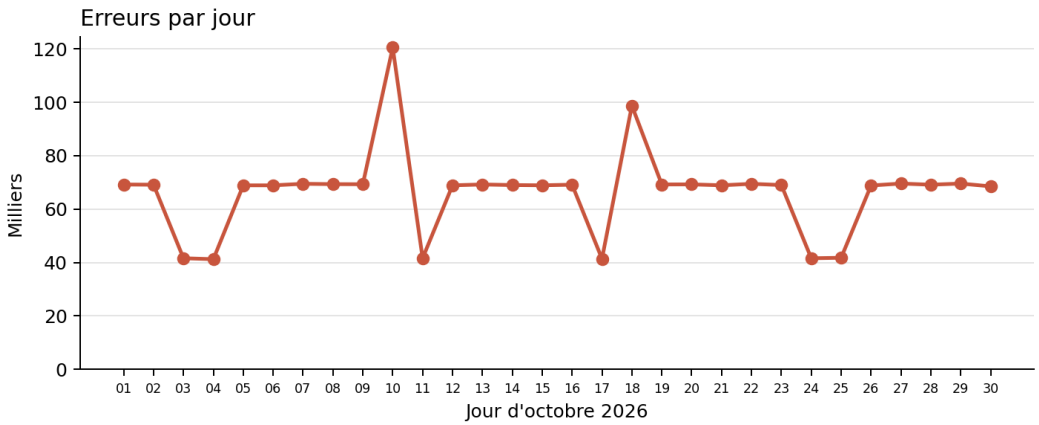
Vérification du tri. Lors de ce test, la place manquait sur le disque : `web.log` a été calculé puis jeté au lieu d'être conservé, et son nombre de lignes vient du compteur du programme. Les 15 autres fichiers ont été recomptés ligne par ligne. Ils correspondent exactement au tableau, chaque ligne est dans le bon fichier et aucun mot de passe n'y apparaît en clair. Les 16 fichiers, `web.log` compris, ont été vérifiés de la même façon sur un jeu plus petit de 0,1 Go.

Relecture. Un fichier trié peut être relu par le programme. Nous avons relu 300 000 lignes de chacun des 15 fichiers conservés : aucune n'est rejetée et les fichiers réécrits sont identiques octet par octet.

Analyse de l'activité

Les données analysées contiennent 172 779 IP actives et 20 014 utilisateurs. Les graphiques font ressortir les événements prévus dans le générateur : la baisse du trafic le week-end, le pic du 18 octobre et la panne du 10 octobre.





6. Résultats de la détection

Pour chaque scénario, nous comparons les entités alertées à celles du manifeste. La **précision** indique la part d'alertes justes. Le **rappel** indique la part d'attaques retrouvées.

Scénario	Attendus	Détectés	Faux positifs	Précision	Rappel
Force brute rapide	180	240	60	75,0 %	100,0 %
Force brute lente	60	60	0	100,0 %	100,0 %
Compte compromis	46	46	0	100,0 %	100,0 %
Pulvérisation	90	90	0	100,0 %	100,0 %
Scan de ports	149	149	0	100,0 %	100,0 %
Attaque web	240	240	0	100,0 %	100,0 %
Exfiltration	60	60	0	100,0 %	100,0 %
Voyage impossible	90	90	0	100,0 %	100,0 %
Liste noire	360	360	0	100,0 %	100,0 %
Routeur reconfiguré	60	60	0	100,0 %	100,0 %

Le faux positif est volontaire. Les 60 faux positifs de la force brute rapide sont les comptes de service au mot de passe expiré : ils échouent plusieurs fois de suite, comme un attaquant. Pour les distinguer, il faudrait une liste des comptes de service. Nous préférons le signaler ici plutôt que de masquer le problème.

Le scanner interne n'est pas signalé, car son adresse est dans `ALLOWED_IPS`. Il démontre le filtrage d'une exception connue.

La force brute lente échappe à la fenêtre de 120 secondes, car les essais sont espacés de plusieurs minutes. Un second détecteur, avec une fenêtre de 6 heures, la repère.

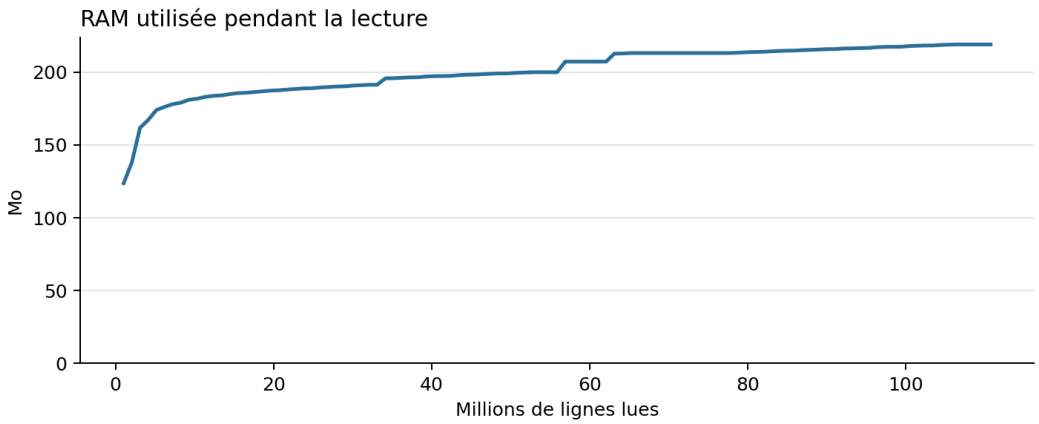
Ce score parfait sur le rappel a une limite : les attaques sont celles que nous avons nous-mêmes programmées, avec des seuils adaptés. Il prouve que le code fonctionne, pas qu'il détecterait n'importe quelle attaque réelle.

Le fichier `alerts.jsonl` contient 42 599 alertes. Une même règle ne se redéclenche pas sur la même entité pendant une heure, pour éviter de noyer l'analyste.

7. Performance et mémoire

Mesure	Valeur
Taille du fichier	15,2 Go (16,3 milliards d'octets)
Lignes lues	111 047 646
Durée	61,6 minutes
Débit	30 044 lignes/s, 4,4 Mo/s
Pic de RAM	219 Mo
Taille d'un lot	10 000 lignes
Machine de test	2 cœurs, 7,8 Go de RAM, Python 3.13

Le fichier fait environ deux fois la RAM de la machine : il serait impossible de le charger avec `read()`. Sur les dix premiers millions de lignes, la RAM monte de 124 à 182 Mo, car le programme découvre la plupart des adresses IP et remplit ses compteurs. Elle ne gagne ensuite que 37 Mo sur les cent millions de lignes suivantes, car les nouvelles adresses deviennent rares.



Le programme tourne sur un seul cœur. Le débit est limité par le calcul : plusieurs fonctions Python sont appelées pour chaque ligne. Le disque n'est pas le frein, car le générateur écrit le même fichier cinq fois plus vite. La durée mesurée ne comprend pas l'écriture de `web.log` sur le disque (section 5).

8. Environnement et qualité du code

• **Environnement :** le projet tourne dans un venv. `requirements.txt` vient de `pip freeze`. Nous l'avons réinstallé dans un environnement neuf sur Python 3.10, 3.13 et 3.14. numpy et contourpy ont une ligne conditionnelle, car aucune de leurs versions n'existe pour 3.10 et 3.14 à la fois.

• **Dépendances :** le pipeline n'utilise que la bibliothèque standard et psutil, qui mesure la RAM. matplotlib trace les graphiques.

• **PEP8 :** flake8 ne signale aucune erreur, avec la limite de 79 caractères par ligne.

• **Typage :** toutes les fonctions ont leurs paramètres et leur retour annotés.

• **Exceptions :** les fichiers, le JSON, les dates et les règles sont protégés par `try/except`. Une ligne invalide ne peut pas arrêter le traitement.

• **Tests :** 28 tests vérifient chaque module, la ligne de commande, le tri et un passage complet comparé au manifeste.

• **Git :** le code est versionné sur un dépôt Git. Les données générées en sont exclues : l'option `--generate` les recrée à l'identique avec la même graine.

9. Limites et améliorations

• **Données synthétiques.** Les résultats valent pour nos scénarios. Un vrai réseau produirait des attaques et des faux positifs différents.

• **Ordre du temps.** La détection suppose que le fichier est à peu près chronologique. C'est le cas de notre fichier. Le tri par date d'un fichier mélangé demanderait une autre méthode, car un tri classique charge tout en mémoire.

• **Place sur le disque.** Les fichiers triés prennent presque autant de place que le fichier de départ. Il faut prévoir 30 Go libres, ou utiliser `--no-sort`.

• **IPv4 uniquement.** Une adresse IPv6 serait rejetée en quarantaine.

• **Temps réel simulé.** `python main.py --simulate` alimente le moteur avec un générateur, sans fichier. La même chaîne pourrait lire une connexion réseau, ce que nous n'avons pas fait.

• **Un seul cœur.** Traiter un fichier par processus accélérerait le calcul, mais ce n'était pas nécessaire ici et cela sort du cours.

• **Longues lignes.** Une ligne géante sans retour à la ligne serait lue en entier. Une lecture avec `readline(limite)` protégerait la mémoire.

10. Reproduire les résultats

```
python -m venv .venv
.venv\Scripts\activate            (Windows)
source .venv/bin/activate         (Linux, macOS)
pip install -r requirements.txt

python main.py                    démonstration sur le jeu de test
python tests/test_project.py      tests

python main.py --generate=data/logs/logs.log
python main.py --log-file=data/logs/logs.log --output=output/complet
python main.py --charts=output/complet
```

La graine étant fixée à 42, ces commandes recréent le même fichier. La durée et la RAM dépendent de la machine. Pour une démonstration rapide, `--max-lines=2000000` arrête la lecture après deux millions de lignes.

Annexe : correspondance avec l'énoncé et le cours

Chaque notion demandée par l'énoncé, sa place dans le code et les diapositives du cours correspondantes.

Exigence	Dans le code	Cours
Ligne de commande	sys.argv, parse_options	CR5, 51 (module sys)
yield, générateurs	read_log_file, make_batches, count_stage	CR4, 27 à 31
yield from	read_sources, simulated_stream	CR4, 32
Lecture d'un gros fichier	read_log_file	CR6, 38
Tuples	record["meta"], clés (ip, compte)	CR3, 24 à 30
filter, filter(None), lambda	build_record_stream	CR4, 18, 24 à 26
map	build_record_stream	CR4, 22 et 23
f-strings	record["tag"]	CR1, 39 et 40
match-case	route_record, format_payload	CR2, 18 à 22
set, | & - ^	compare_with_blacklist	CR3, 32 à 42
frozenset	load_blacklist	CR3, 43
defaultdict	Aggregator	CR3, 55 et 56
while, for	BruteForceDetector.run_cycle	CR2, 24 à 27
break, continue, else	run_cycle	CR2, 28 à 31
*args, **kwargs	apply_security_rules	CR4, 8 à 11, 15 et 16
Fonctions en argument	Règles, callback publish	CR4, 37 à 40
globals(), locals()	run_pipeline, audit.py	CR4, 35 et 36
Classes	Aggregator, BruteForceDetector, LogSorter	CR5, 17 à 22
Package	logengine/__init__.py	CR5, 52 à 54
Exceptions, with, encodage	ingestion.py, parsing.py	CR6, 24 à 28, 36 et 37
json, csv	pipeline.py	CR3, 64 et CR6, 29 à 33
venv, requirements.txt	requirements.txt	CR1, 13 à 16
Graphiques	charts.py	CR6, 42 à 54